Mini-Project Assignment

Flower Shop Management System using Dictionary

Problem Statement

Design and implement a **menu-driven Python application** for managing a **Flower Shop Inventory System** using the **dictionary data structure**.

The system should maintain information about flowers available in the shop. Each flower record must store the following details:

- **Flower Name**
- **Price per bunch**
- **Available Quantity**
- **Category / Type** *(optional: Rose, Lily, Bouquet, Decorative, Seasonal, etc.)*

All flower records must be stored and managed using a **Python dictionary**. Each operation in the system must be implemented as a **separate function**, and these functions should be invoked through a **menu-driven program**.

Data Structure Requirement

The flower data should be stored in a dictionary using one of the following formats.

Option 1: Simple Dictionary

```
flowers = {
    "Rose": {"price": 50, "quantity": 20},
    "Lily": {"price": 80, "quantity": 15},
    "Tulip": {"price": 100, "quantity": 10}
}
```

Option 2: Extended Dictionary

```
flowers = {
    "Rose": {"price": 50, "quantity": 20, "category": "Decorative"},
    "Lily": {"price": 80, "quantity": 15, "category": "Seasonal"}
}
```

Functional Requirements

Your program must implement the following functions:

1. Add a New Flower

Create a function to insert a new flower record into the dictionary.

The function should:

- Accept flower name
- Accept price
- Accept quantity
- Store the data in the dictionary

2. Update Flower Details

Create a function to modify the **price or quantity** of an existing flower.

The function should:

- Search flower by name
- Update price and/or quantity

3. Delete a Flower Record

Create a function to remove a flower entry from the dictionary.

4. Search for a Flower

Create a function to search for a flower by name and display its details.

5. Display All Flowers

Create a function to display all flower records in a well-formatted table.

6. Display Only Flower Names

Create a function to display all **flower names** stored in the dictionary.

7. Display Only Prices or Quantities

Create a function to display **only the values** stored in the dictionary (price and quantity).

8. Check Whether a Flower Exists

Create a function to check whether a given flower exists in the shop using the **dictionary membership operator (`in`)**.

9. Count Total Flower Types

Create a function to display the **total number of flowers** available in the dictionary.

10. Find the Most Expensive Flower

Create a function to identify the flower having the **maximum price**.

11. Find the Cheapest Flower

Create a function to identify the flower having the **minimum price**.

12. Calculate Total Stock Value

Create a function to compute the total value of all flowers in stock.

Formula:

[
Total\ Stock\ Value = price \times quantity
]

The function should calculate the total value of each flower and then display the **grand total**.

13. Display Flowers with Low Stock

Create a function to display flowers whose quantity is **below a given threshold**.

Example:
Display all flowers whose quantity is **less than 5**.

14. Sort Flowers by Name

Create a function to display flower records sorted **alphabetically by flower name**.

15. Sort Flowers by Price

Create a function to display flower records sorted **by price** in ascending or descending order.

16. Sell Flower (Reduce Quantity)

Create a function to simulate selling flowers.

The function should:

- Accept flower name
- Accept quantity sold
- Reduce available quantity

Conditions:

- If enough quantity exists → update stock
- If not → display **"Insufficient stock"**

17. Restock Flower

Create a function to increase the quantity of a flower.

18. Create a Dictionary of Expensive Flowers

Using **dictionary comprehension**, create a new dictionary containing flowers whose **price is above a specified amount**.

19. Clear All Flower Records

Create a function to remove all flower records from the dictionary.

20. Exit the Program

Terminate the program when the user selects the exit option.

Dictionary Operations That Must Be Used

Your program must demonstrate the use of the following **dictionary concepts and functions**:

- dictionary creation
- nested dictionary
- adding key-value pairs
- updating values
- deleting records
- searching keys
- `in` operator
- `len()`
- `keys()`
- `values()`
- `items()`
- `get()`
- `del`
- `pop()`
- `clear()`
- `sorted()`
- `max()`
- `min()`
- dictionary comprehension

Menu-Driven Requirement

The main program must repeatedly display the following menu:

```
========= Flower Shop Management =========
1. Add Flower
2. Update Flower
3. Delete Flower
4. Search Flower
5. Display All Flowers
6. Display Flower Names
7. Display Flower Details
8. Check Flower Availability
9. Count Total Flower Types
10. Find Most Expensive Flower
11. Find Cheapest Flower
12. Calculate Total Stock Value
13. Display Low Stock Flowers
14. Sort Flowers by Name
15. Sort Flowers by Price
16. Sell Flower
17. Restock Flower
18. Show Expensive Flowers
19. Clear All Records
20. Exit
Enter your choice:
```

The program must continue running until the user selects **Exit**.